

Statement Purpose:

Objective of this lab is to learn how to define Interface and implement Interfaces. Students will also learn how to implement multiple interfaces in one class.

Activity Outcomes:

This lab teaches you the following topics:

- Interface class.
- Implementation of Interface classes and its methods.
- Implementation of multiple interfaces in class.
- Use of interfaces with abstract classes

Instructor Note:

As pre-lab activity, read Chapter 7 from the book, (Absolute Java, Savitch, W. & Mock, K., 5th Edition (2012), Pearson.), and also as given by your theory instructor.

1) Stage I (Journey)

Introduction

An interface is a type that groups together a number of different classes that all include method definitions for a common set of method headings.

Java interface specifies a set of methods that any class that implements the interface must have. An interface is itself a type, which allows you to define methods with parameters of an interface type and then have the code apply to all classes that implement the interface. One way to view an interface is as an extreme form of an abstract class. However, as you will see, an interface allows you to do more than an abstract class allows you to do. Interfaces are Java's way of approximating multiple inheritance. You cannot have multiple base classes in Java, but interfaces allow you to approximate the power of multiple base classes.

Syntax:

```
How to define interface public  
  
interface Ordered { public boolean  
  
    precedes(Object other);  
  
}
```

How to implement interface and its method public

```
class Class_name implements Interface_Name { public  
  
boolean precedes(Object other){  
  
}}  
  
public class implements SomeInterface, AnotherInterface {}
```

Abstract Classes Implementing Interfaces

A concrete class (that is, a regular class) must give definitions for all the method headings given in an interface in order to implement the interface. However, you can define an abstract class that implements an interface but gives only definitions for some of the method headings given in the interface. The method headings given in the interface that are not given definitions are made into abstract methods.

Derived Interfaces

You can derive an interface from a base interface. This is often called extending the interface. The details are similar to deriving a class.

The Comparable Interface

The Comparable interface is in the java.lang package and so is automatically available to your program. The Comparable interface has only the following method heading that must be given a definition for a class to implement the Comparable interface:

```
public int compareTo(Object other);
```

The method compareTo should return a negative number if the calling object —comes before the parameter other, a zero if the calling object —equals the parameter other, and a positive number if the calling object —comes after the parameter other. The —comes before ordering that underlies compareTo should be a total ordering. Most normal ordering, such as less-than ordering on numbers and lexicographic ordering on strings, is total ordering.

Defined Constants in Interfaces

The designers of Java often used the interface mechanism to take care of a number of miscellaneous details that do not really fit the spirit of what an interface is supposed to be. One example of this is the use of an interface as a way to name a group of defined constants. An interface can contain defined constants as well as method headings, or instead of method headings. When a method implements the interface, it automatically gets the defined constants. For example, the following interface defines constants for months:

```
public interface MonthNumbers {
```

```
public static final int JANUARY = 1, FEBRUARY = 2, MARCH = 3, APRIL = 4,
MAY = 5, JUNE = 6, JULY = 7, AUGUST = 8, SEPTEMBER = 9,
OCTOBER = 10, NOVEMBER = 11, DECEMBER = 12; }
```

Any class that implements the MonthNumbers interface will automatically have the 12 constants defined in the MonthNumbers interface. For example, consider the following toy class:

```
public class DemoMonthNumbers implements MonthNumbers {

    public static void main(String[] args) {

        System.out.println( "The number for January is " + JANUARY);    } }
```

2) Stage **a1** (apply)

Lab Activities:

Activity 1:

Declare a Interface, RegisterForExams that contains single method register, implements the interface in two different classes (a) Student (b) Employee. Write down a Test Application that contains at least a function that takes Interface type Parameter

Solution:

```
public interface RegisterForExams {
    public void register();
}

public class InterfaceTestClass {

    public InterfaceTestClass(RegisterForExams as)
    {
        as.register();
    }
}

public class EmployeeTask implements RegisterForExams{

    private String name;    private
String date;    private int salary;

    public EmployeeTask()
    {
        name = null;
date = null;
        salary = 0;
    }

    public EmployeeTask(String name,String date,int salary)
```

```

        {
            this.name = name;
this.date = date;
            this.salary = salary;
        }

        @Override
        public void register() {
            // TODO Auto-generated method stub
            System.out.println("Name " + name + "\nsalary " + salary + "\n Employee reistered on date " +
date);
        }
    }
public class StudentTask implements RegisterForExams{
        private String name;          private int
age;
        private double gpa;

        public StudentTask()
        {
            name = null;
            age = 0;
            gpa = 0;
        }
        public StudentTask(String name,int age,double gpa)
        {
            this.name = name;
this.age = age;          this.gpa = gpa;
        }

        @Override
        public void register() {
            // TODO Auto-generated method stub
            System.out.println("Student name " + name + " gpa " + gpa);
        }
    }

```

Activity 2:

This example show how to implement interface to a class `public class
OrderedHourlyEmployee extends HourlyEmployee implements Ordered{`

```

public boolean precedes(Object other) {
    if (other == null)
return false;
    else if (!(other instanceof OrderedHourlyEmployee)) return
false;

    else {

        OrderedHourlyEmployee otherOrderedHourlyEmployee = (OrderedHourlyEmployee)other;
return (getPay() < otherOrderedHourlyEmployee.getPay());
    }
}

```

```

    }
}
public boolean follows(Object other)
{ if (other ==
null) return
false;
else if (!(other instanceof OrderedHourlyEmployee))

return false; else
{
OrderedHourlyEmployee otherOrderedHourlyEmployee = (OrderedHourlyEmployee)other;
return (otherOrderedHourlyEmployee.precedes( this));
} }
}

```

Activity 3:

An Example that shows How to create your own interface and implement it in abstract class

```

interface I1 {

void methodI1(); // public static by default
}

interface I2 extends I1 {

void methodI2(); // public static by default
}

class A1 {

public String methodA1() {
String strA1 = "I am in methodC1 of class A1";      return
strA1;
}
public String toString() {
return "toString() method of class A1";
}
}

class B1 extends A1 implements I2 {

public void methodI1() {
System.out.println("I am in methodI1 of class B1");
}
public void methodI2() {
System.out.println("I am in methodI2 of class B1");
}
}

class C1 implements I2 {

```

```

public void methodI1() {
    System.out.println("I am in methodI1 of class C1");
}
public void methodI2() {
    System.out.println("I am in methodI2 of class C1");
}
}

// Note that the class is declared as abstract as it does not
// satisfy the interface contract abstract
class D1 implements I2 {

    public void methodI1() { }
    // This class does not implement methodI2() hence declared abstract. }

public class InterFaceEx {

    public static void main(String[] args) {
        I1 i1 = new B1();
        i1.methodI1(); // OK as methodI1 is present in B1
        // i1.methodI2(); Compilation error as methodI2 not present in I1
        // Casting to convert the type of the reference from type I1 to type I2
        ((I2) i1).methodI2();    I2 i2 =
new B1();    i2.methodI1();
// OK    i2.methodI2(); // OK
        // Does not Compile as methodA1() not present in interface reference I1
        // String var = i1.methodA1();
        // Hence I1 requires a cast to invoke methodA1
        String var2 = ((A1) i1).methodA1();    System.out.println("var2 : " +
var2);

        String var3 = ((B1) i1).methodA1();
        System.out.println("var3 : " + var3);
        String var4 = i1.toString();
        System.out.println("var4 : " + var4);
        String var5 = i2.toString();
        System.out.println("var5 : " + var5);
        I1 i3 = new C1();
        String var6 = i3.toString();
        System.out.println("var6 : " + var6); // It prints the Object toString() method
        Object o1 = new B1();
        // o1.methodI1(); does not compile as Object class does not define
        // methodI1()
        // To solve the problem we need to downcast o1 reference. We can do it
        // in the following 4 ways
        ((I1) o1).methodI1(); // 1
        ((I2) o1).methodI1(); // 2
        ((B1) o1).methodI1(); // 3
    }
}
/*
*

```

* B1 does not have any relationship with C1 except they are "siblings".

```

        *
    * Well, you can't cast siblings into one another.
        *
    */
    // ((C1)o1).methodI1(); Produces a ClassCastException
}
}

```

3) Stage **V** (verify)

Home Activities:

Activity 1:

Create an interface EnhancedShape that extends Shape and also requires a method public double perimeter() to be implemented by any class that uses the interface.

Activity 2:

Write down a test application that implements EnhancedShape interface, call the interface methods to verify the functionality of implemented methods.

Activity 1:

Code Conflicting interfaces i.e. containing same constants with different values or exactly same method signatures in different interfaces, note down what happens if you try to implement conflicting interfaces.

4) Stage **a2** (assess)

Lab Assignment 3 and Viva voce [Marks 10]

Task 1

Define an interface named Shape with a single method named area that calculates the area of the geometric shape: public double area();

Task 2

Implement the Shape interface for Rectangle, Circle and Triangle class.

Task 3

Implement a class CalculateAreas that has a function that takes shape type array of objects and builds an array of (double values) values for each corresponding shapes.